The Seat component calls useContext a second time to get access to the subscription context:

chapter_11/02/app/javascript/components/seat.tsx

```
const context = React.useContext<IsVenueContext>(VenueContext)
const subscription = React.useContext<Subscription>(SubscriptionContext)
```

Then it sends a command back to the subscription, almost exactly as before:

chapter_11/02/app/javascript/components/seat.tsx

```
const onSeatChange = (): void => {
  const status = currentStatus()
  if (status === "invalid" || status === "purchased") {
    return
  }
  const actionType = status === "unsold" ? "holdTicket" : "unholdTicket"
  context.dispatch({ type: actionType, seatNumber, rowNumber })
  subscription.perform("added_to_cart", {
    concertId: context.state.concertId,
    row: rowNumber,
    seatNumber: seatNumber,
    status: actionType === "holdTicket" ? "held" : "unsold",
    ticketsToBuyCount: context.state.ticketsToBuyCount,
  })
}
```

Similarly, the Subtotal component needs to tell the server side that all the personally held tickets can be moved to the unsold state when the page is cleared:

chapter_11/02/app/javascript/components/subtotal.tsx

```
const subscription = React.useContext<Subscription>(SubscriptionContext)

const onClear = () => {
  subscription.perform("removed_from_cart", {
    concertId: context.state.concertId,
    tickets: context.state.myTickets,
  })
  context.dispatch({ type: "clearHolds" })
}
```